OpenMP — Lesson 6

Coordinating the team: `barrier`, `single` and `sections`

Gustavo Lozano

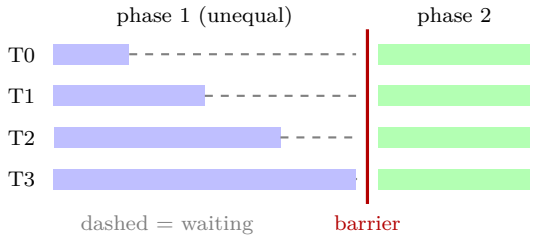
November 2026

Source: Chapman, Jost & van der Pas [1], Ch. 4 "OpenMP Language Features" (the `single` and `sections` work-sharing constructs and the `barrier` synchronization construct).

1 The idea

Lesson 4 split a *loop* among threads. But a parallel region often contains other kinds of work:

- a step that must be done **once**, not by every thread (read input, fill an array, print a result) → `single`;
- a few **different**, independent tasks that can run at the same time → `sections`;
- a point that **all** threads must reach before any continues, because the next phase needs the results of the previous one → `barrier`.



2 The three constructs

Construct	What it does	Barrier at the end?
`#pragma omp barrier`	every thread waits here until all threads of the team arrive	it *is* a barrier
`#pragma omp single`	the block is run by exactly **one** thread (any one); the others skip it	yes (removable with `nowait`)
`#pragma omp sections` with several `#pragma omp section`	each `section` is a different block, run exactly once by some thread of the team	yes (removable with `nowait`)

`single` and `sections` are **work-sharing** constructs, like `for`: they divide work among the threads that already exist, and do not create new ones. They must appear inside a `parallel` region.

3 The code

Part 1 gives each thread an unequal phase 1 (thread k sleeps $0.1k$ seconds), then a barrier, then phase 2. Part 2 uses `single` to fill an array once, two `sections` to compute its sum and its maximum, and a second `single` to print the result.

single_sections.c

```c
/*
 * single_sections.c -- OpenMP Lesson 6: barrier, single and sections.
 * Source: Chapman, Jost & van der Pas (2007), "Using OpenMP",
 *         Ch. 4 (single and sections work-sharing constructs,
 *         barrier synchronization construct).
 *
 * Compile:
 *     gcc -Wall -fopenmp -o single_sections single_sections.c
 * Run:
 *     OMP_NUM_THREADS=4 ./single_sections
 */

#include <stdio.h>
#include <unistd.h>     /* usleep(): pause a thread for some microseconds */
#include <omp.h>

#define N 100

int main(void)
{
    int  a[N];
    long sum = 0;
    int  max = 0;

    /* ================= Part 1: explicit barrier ================= */
    printf("--- Part 1: barrier ---\n");
    #pragma omp parallel
    {
        int id = omp_get_thread_num();

        usleep(100000 * id);              /* phase 1: thread id works id*0.1 s */
        printf("  thread %d finished phase 1\n", id);

        #pragma omp barrier
        /* No thread passes this line until ALL threads have reached it. */

        printf("  thread %d started  phase 2\n", id);
    }

    /* ============== Part 2: single and sections ================= */
    printf("--- Part 2: single and sections ---\n");
    #pragma omp parallel
    {
        /* single: exactly ONE thread (whichever arrives first) runs the
           block; the others skip it and WAIT at its implicit barrier,
           so nobody reads a[] before it is filled.                    */
        #pragma omp single
        {
            printf("  thread %d fills the array (single)\n", omp_get_thread_num());
            for (int i = 0; i < N; i++)
                a[i] = (i * 37) % 101;    /* some numbers between 0 and 100 */
        }

        /* sections: DIFFERENT blocks of code, each run exactly once. Free
           threads take the sections; which thread gets which is up to the
           runtime (one thread may even run both if they are short).
           Here: two independent tasks on the same data. Threads without
           a section just wait at the implicit barrier.                  */
        #pragma omp sections
        {
            #pragma omp section
            {
                for (int i = 0; i < N; i++)
```

```
                    sum += a[i];          /* only this section writes sum */
                printf("  thread %d computed sum (section 1)\n", omp_get_thread_num());
            }
            #pragma omp section
            {
                for (int i = 0; i < N; i++)
                    if (a[i] > max) max = a[i];   /* only this section writes max */
                printf("  thread %d computed max (section 2)\n", omp_get_thread_num());
            }
        }   /* implicit barrier: both results are ready after this line */

        #pragma omp single
        printf("  thread %d prints: sum = %ld, max = %d (single)\n",
               omp_get_thread_num(), sum, max);
    }
    return 0;
}
```

```
gcc -Wall -fopenmp -o single_sections single_sections.c
OMP_NUM_THREADS=4 ./single_sections
```

Typical output (thread numbers and order will differ):

```
--- Part 1: barrier ---
  thread 0 finished phase 1
  thread 1 finished phase 1
  thread 2 finished phase 1
  thread 3 finished phase 1
  thread 3 started  phase 2
  thread 1 started  phase 2
  thread 0 started  phase 2
  thread 2 started  phase 2
--- Part 2: single and sections ---
  thread 3 fills the array (single)
  thread 2 computed sum (section 1)
  thread 2 computed max (section 2)
  thread 3 prints: sum = 4986, max = 100 (single)
```

Key ideas

- With the barrier, *all* "finished phase 1" lines come before *any* "started phase 2" line, even though thread 0 finished 0.3 s before thread 3.
- `single`: the array is filled once. Its implicit barrier is what guarantees that no thread starts reading `a[]` before it is full.
- `sections`: each section runs exactly once, but you do not choose the thread. Short sections are often both taken by the same thread, as in the output above. Sections guarantee *once*, not *in parallel*.
- No race on `sum` or `max`: each is written by only one section, and the final `single` reads them after the barrier of `sections`.

4 The experiment

Experiment

Question: what does the barrier actually guarantee?

1. Run the program as it is, several times, and look at Part 1:

```
for r in 1 2 3; do OMP_NUM_THREADS=4 ./single_sections | head -9; echo; done
```

2. Comment out the barrier in Part 1, recompile, and run again:

```
        /* #pragma omp barrier */
```

3. In the same runs, note which thread executes each `single` and each `section` in Part 2.

What to observe and answer.

1. Without the barrier, how are the phase 1 and phase 2 lines ordered now? Why?
2. Suppose phase 2 read a value that *every* thread wrote in phase 1. What would go wrong without the barrier?
3. Is the thread that runs a `single` always the same? Is it always thread 0?

Answers (read after trying). (1) Each thread starts phase 2 right after its own phase 1, so the lines appear in pairs (thread 0 finishes and starts, then thread 1, . . .). Nothing makes a thread wait for the others. (2) A fast thread would start phase 2 while slow threads are still writing their phase 1 values, and would read incomplete data. That is a race between phases, and it is exactly what a barrier prevents. (3) No. The standard leaves the choice to the implementation; in practice it is usually the first thread to arrive, which changes from run to run. If the work must be done by the master thread, OpenMP has a separate construct, `master` (renamed `masked` in recent versions), which, unlike `single`, has *no* implicit barrier.

Next lesson. Tasks: the `task` and `taskwait` constructs, for work that is not a loop and whose size is not known in advance (van der Pas et al. [2], chapter on tasking).

References

[1] B. Chapman, G. Jost, and R. van der Pas. *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press, Cambridge, MA, 2007.

[2] R. van der Pas, E. Stotzer, and C. Terboven. *Using OpenMP — The Next Step: Affinity, Accelerators, Tasking, and SIMD*. MIT Press, Cambridge, MA, 2017.